

Day-67-JSON Data in Python

1. Day Number

Day 67

2. Topic Name

JSON and Python Dictionaries

3. Connection

You already understand **Python dictionaries** and have started working with **structured files** such as CSV and Excel.

Today you will learn how dictionary-like data can be stored permanently in a **JSON file**.

A simple flow is:

```
Python dictionary
    ↓
Save as JSON
    ↓
JSON file
    ↓
Read JSON
    ↓
Python dictionary again
```

4. Important Topics

Today, focus on these concepts:

- JSON
- JSON object
- key-value pair
- JSON list
- Python json module
- `json.dump()`
- `json.load()`

For now, keep the JSON structure simple. We will avoid complex nested data.

5. Foundational Notes

What is JSON?

JSON stands for:

JavaScript Object Notation

Despite the name, JSON is used by many programming languages, including Python.

JSON is a common format for storing and exchanging structured data.

For example:

```
{  
  "name": "Amit",  
  "city": "Bengaluru"  
}
```

This looks very similar to a Python dictionary.

JSON files normally use the extension:

```
.json
```

For example:

```
customer.json
```

JSON Object

A JSON object stores information using **key-value pairs**.

```
{  
  "name": "Amit",  
  "city": "Bengaluru"  
}
```

Here:

```
"name" → key  
"Amit" → value  
  
"city" → key  
"Bengaluru" → value
```

JSON List

JSON can also contain lists:

```
{  
  "cities": ["Bengaluru", "Mumbai", "Delhi"]  
}
```

This is similar to a Python list.

For Day 67, you mainly need to understand that JSON supports objects and lists. You do not need complicated combinations yet.

6. JSON vs Python Dictionary

They look similar, but they are not exactly the same thing.

Python dictionary

A Python dictionary exists inside your Python program:

```
customer = {  
    "name": "Amit",  
    "city": "Bengaluru"  
}
```

JSON

JSON is a **data format** that can be stored in a file:

```
{  
    "name": "Amit",  
    "city": "Bengaluru"  
}
```

A useful way to remember the difference:

Python Dictionary	JSON
Python data structure	Data-storage format
Exists inside Python code	Can be stored in a .json file
Uses Python syntax	Uses JSON syntax
Can contain Python-specific values	Supports standard JSON data types
Used while program runs	Often used to save or exchange data

One visible difference appears with Boolean values.

Python:

```
True  
False  
None
```

JSON:

```
true  
false  
null
```

Python's json module handles these conversions automatically.

7. Easy Example

Suppose you have customer information:

```
customer = {  
    "name": "Ravi",  
    "city": "Mysuru"  
}
```

You want to save this information into:

```
customer.json
```

Python provides the built-in json module:

```
import json
```

To **write Python data to JSON**, you will use:

```
json.dump(...)
```

To **read JSON data into Python**, you will use:

```
json.load(...)
```

Think of the names like this:

```
dump → Python data → JSON file  
load → JSON file → Python data
```

You do **not** need to install anything.

The json module comes with Python.

8. Problem Statement

Create a program that stores simple customer information.

The customer should have:

```
name  
city
```

Your program should:

1. Create customer data using a Python dictionary.
2. Open a JSON file for writing.
3. Save the dictionary into the JSON file.
4. Open the same JSON file for reading.
5. Read the JSON data back into Python.
6. Print the customer's name.
7. Print the customer's city.

Example data:

```
Name: Ananya  
City: Bengaluru
```

Do not create complex nested customer information yet.

9. Concepts Used

You will practice:

```
dictionary  
key-value pair  
file handling  
with statement  
read mode  
write mode  
import  
json module  
json.dump()  
json.load()  
dictionary key access  
functions
```

Notice how this combines concepts from earlier days.

You already know dictionaries:

```
customer["name"]
```

Now you are learning how to **save that dictionary into a file**.

10. Thought Process

Before writing code, think about the program in stages.

Step 1: What data do I have?

You need two pieces of information:

```
name  
city
```

A dictionary is suitable because each value has a meaningful key.

Conceptually:

```
customer = {  
    "name": "...",  
    "city": "..."  
}
```

Step 2: How will I store it?

Use a JSON file:

```
customer.json
```

The dictionary needs to be converted into JSON format.

That is what:

```
json.dump()
```

helps with.

Step 3: How will I retrieve it?

Open the JSON file again and use:

```
json.load()
```

The JSON object will become a Python dictionary.

Step 4: How will I display the information?

Access the dictionary values using their keys:

```
name  
city
```

Then print them.

So the complete mental model is:

```
Create dictionary
  ↓
Open file
  ↓
json.dump()
  ↓
customer.json
  ↓
Open file
  ↓
json.load()
  ↓
Dictionary
  ↓
Print values
```

11. Beginner-Friendly Pseudocode

Do not worry about exact Python syntax yet.

```
START

import JSON module

create customer dictionary
    name = Ananya
    city = Bengaluru

create a function to save customer data

    open customer.json in write mode

    save dictionary using json.dump()

create a function to read customer data

    open customer.json in read mode

    load data using json.load()

    return loaded data

call save function

call read function
```

```
get customer name

get customer city

print name
print city

END
```

12. Suggested Solving Approach: Functional Approach

Use small functions instead of placing everything together.

For example, conceptually:

```
save_customer(...)
read_customer(...)
```

The first function should be responsible for:

```
Python dictionary → JSON file
```

The second function should be responsible for:

```
JSON file → Python dictionary
```

Then your main program can do something like:

```
create data
save data
read data
print data
```

This separation makes the program easier to understand and test.

A possible structure is:

```
import json

def save_customer(...):
    # save the dictionary

def read_customer(...):
    # read and return the dictionary

# create customer
```

```
# save customer  
  
# read customer  
  
# print values
```

This is only the **structure**, not the complete solution.

13. Easy Edge Cases

Edge Case 1: Empty JSON File

Imagine:

```
customer.json
```

exists but contains nothing.

Then trying to use:

```
json.load(...)
```

will cause an error because an empty file is not valid JSON.

Later you can learn exception handling for situations like this.

For now, simply remember:

```
Empty file ≠ valid JSON
```

A valid empty JSON object would instead look like:

```
{}
```

Edge Case 2: Missing Key

Imagine the file contains:

```
{  
    "name": "Ananya"  
}
```

There is no:

```
city
```

If your program assumes "city" always exists and tries to access it directly, it can cause an error.

One beginner-friendly idea is dictionary `.get()`:

```
customer.get("city")
```

You can also provide a default value:

```
customer.get("city", "Unknown")
```

Conceptually:

```
If city exists → return the city
If city does not exist → return "Unknown"
```

14. Expected JSON and Output

After saving your data, `customer.json` might contain:

```
{
  "name": "Ananya",
  "city": "Bengaluru"
}
```

Your Python program should eventually produce output similar to:

```
Customer name: Ananya
Customer city: Bengaluru
```

The exact spacing or wording is not important.

The important part is that the program successfully performs:

```
dictionary
  ↓
JSON file
  ↓
dictionary
  ↓
printed values
```

15. Hint Only

Remember these two operations:

```
json.dump(data, file)
```

means:

```
save Python data → JSON file
```

and:

```
json.load(file)
```

means:

```
read JSON file → Python data
```

Your main challenge is deciding **which file mode** to use for each operation:

```
Saving → ?
```

```
Reading → ?
```

Also think about this sequence:

```
import json

create dictionary

define save function
    open file in correct mode
    use json.dump()

define read function
    open file in correct mode
    use json.load()
    return data

save

read

print name
print city
```

Try solving it yourself without looking for a complete solution.

Day 67 key takeaway: a Python dictionary can be written to a JSON file using `json.dump()`, and JSON data can be brought back into Python using `json.load()`.